



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Edu Verse AI

AN NLP-POWERED STUDENT GROWTH OPERATING SYSTEM WITH INTELLIGENT DIGITAL TWINS

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of
DSA0303 – Natural Language Processing

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE & DATA SCIENCE

Submitted by

S Hemanth Kumar (192424025)

M Abishek (192424022)

M Teja (192424328)

K V Sai Shanmukh (192424306)

Under the Supervision of

Dr. K V Gnanavel

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “EduVerse AI – The World's First NLP-Powered Student Growth Operating System with Intelligent Digital Twins” has been carried out by HEMANTH KUMAR S (192424025), M ABISHEK (192424022), M TEJA (192424328), and K V SAI SHANMUKH (192424306) under the supervision of Dr K V Gnanavel and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Artificial Intelligence & Data Science programmer at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. T. Devi

Professor

AI & DS

SIMATS Engineering

SIMATS, Chennai - 602105

COURSE FACULTY

Dr K V Gnanavel

Course Faculty

Computer Science & Engineering

SIMATS Engineering

SIMATS, Chennai - 602105

Submitted for the Project work Viva-Voce held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

We, Hemanth Kumar S (192424025), M Abishek (192424022), M Teja (192424328), and V Sai Shanmukh (192424306) of the Department of Artificial Intelligence and Data Science, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “EduVerse AI – The World's First NLP-Powered Student Growth Operating System with Intelligent Digital Twins” is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. All external tools, software, datasets, computational resources, and AI-assisted tools used during the project — including sentence-transformers/all-MiniLM-L6-v2, spaCy, distil BART, Whisper, and Groq LLM inference — have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:05-09-2026

Signature of the Students with Names:

Hemanth Kumar S (192424025)

M Abishek (192424022)

M Teja (192424328)

K V Sai Shanmukh (192424306)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. %
S Hemanth Kumar (192424025)	Backend architecture & Digital Twin write path	Fast API backend, Supa base schema, RLS policies	Integration and security testing (TC-01, TC-03)	Chapters 3 & 4	25%
M Abishek (192424022)	NLP microservice & RAG pipeline	Embedding, NER, pgvector retrieval, AI Tutor	Retrieval and unit testing (TC-02, TC-04)	Chapters 2 & 4	25%
M Teja (192424328)	Frontend & Career/Learning module	Next.js dashboard, resume-job matching logic	Matching and performance testing (TC-05)	Chapters 1 & 5	25%
K V Sai Shanmukh (192424306)	Wellness/Campus module & documentation	Sentiment-trend pipeline, campus assistant	Environment and account testing (TC-06)	Chapters 3 & 5, References	25%

ABSTRACT

Most campus platforms in use today record only what a student is doing — marks, attendance, and logins — while the rich language a student generates daily through assignments, chats, resumes, research papers, and feedback goes largely unread. This creates a real engineering gap: career and research guidance remains generic and manual, motivational or wellbeing shifts that are visible in a student's language are never detected, and campus communication stays scattered across channels a student must search manually. EduVerse AI addresses this gap by treating student-generated language as a first-class signal rather than a byproduct, giving every student a continuously evolving AI Digital Twin, EduDNA, that models learning style, technical skill, communication ability, career direction, research interest, and wellbeing — without issuing any diagnosis — purely from their own text.

The proposed solution is a single reusable NLP pipeline — embed → extract → classify → update — instantiated across four integrated modules: Digital Twin & Academic Intelligence (assignment evaluation, plagiarism checks, summarisation, flashcards and quizzes); Learning & Career Intelligence (learning-style detection and semantic resume-to-job matching); Conversational & Research Intelligence (a Retrieval-Augmented Generation AI Tutor and campus assistant that cites its sources); and Wellness & Campus Intelligence (consent-based communication-pattern nudges). The methodology combines sentence-transformers/all-MiniLM-L6-v2, spaCy, distilBART, and Whisper in a FastAPI NLP microservice; a FastAPI backend that writes to a Supabase (PostgreSQL + pgvector + Row Level Security) Digital Twin store; Neo4j Aura Free for a skill knowledge graph; Upstash Redis for caching; Groq's free-tier LLM for generation; and a Next.js frontend — every component drawn deliberately from free-tier and open-source options to prove the architecture is viable at zero budget.

Phases 1 and 2 of the build are complete and tested. Unit, integration, retrieval, security, and environment testing confirm that Row Level Security correctly isolates each student's Digital Twin and documents, that the RAG pipeline grounds AI Tutor answers in cited source chunks rather than unsupported generation, and that the NLP microservice's five endpoints behave correctly across all four modules. The principal conclusion is that a large, modular, 28-concept platform vision can be reduced to one generalisable NLP pattern and delivered as a genuinely functional prototype entirely on free-tier infrastructure, with Phases 3–5 and production deployment identified as the next engineering milestones.

Keywords: Natural Language Processing, Student Digital Twin, Retrieval-Augmented Generation, Semantic Embeddings, Sentiment Analysis, Free-Tier Architecture

TABLE OF CONTENTS

S.No	Title	Page No
	CERTIFICATE FROM THE SUPERVISOR	
	DECLARATION BY THE CANDIDATE	
	INDIVIDUAL CONTRIBUTION STATEMENT	
	ABSTRACT	
	LIST OF FIGURES	
	LIST OF TABLES	
	LIST OF ABBREVIATIONS	
1	CHAPTER 1: Introduction	10 – 12
2	CHAPTER 2: Literature Review	13 – 14
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	15 – 19
4	CHAPTER 4: Implementation, Testing & Results, Project Management	20 – 24
5	CHAPTER 5: Conclusion & Reflection	25 – 27
	REFERENCES	28
	APPENDICES	29 – 32

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Fig. 1	EduVerse AI System Architecture	17
Fig. 2	Capstone-Scope Module Implementation Status	22

LIST OF TABLES

Table No.	Table Name	Page No.
Table 2.1	Review Existing Work	13
Table 2.2	Comparative Analysis of Existing Approaches	14
Table 3.1	Stakeholder / User Requirements	15
Table 3.2	Functional & Non-Functional Requirements	16
Table 3.3	Constraints & Applicable Standards	16
Table 3.4	Design Specifications	16
Table 3.6	Trade-off Analysis	18
Table 3.7	Sustainability, Ethics, Safety, and Security Evidence	19
Table 4.1	Tools / Software Used	20
Table 4.2	Test Plan & Verification	21
Table 4.3	Specification vs Achieved Results	
Table 4.4	Objective Achievement Mapping	
Table 4.5	Roles and Milestones	
Table 4.6	Cost Table (Free-Tier Build)	

LIST OF ABBREVIATIONS / SYMBOLS

Abbreviation	Description
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
NER	Named Entity Recognition
RLS	Row Level Security
API	Application Programming Interface
LLM	Large Language Model
XP	Experience Points (gamification)

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Most campus platforms available today, such as learning management systems and academic ERPs, are designed to record what a student is doing — marks, attendance, and logins — rather than to understand who the student is becoming. Every day a student generates a large volume of unstructured language: assignments, chat messages with peers, resumes, research papers, emails, and feedback, and virtually none of this language is read or reasoned over by existing systems. Natural Language Processing, applied to this stream of student-generated text, offers a natural technical foundation for a platform that models learning style, technical skill, communication ability, career direction, research interest, and wellbeing over time, continuously updating as the student grows.

1.2 Need for the Project

The problem needs to be addressed because career and research guidance in most institutions is generic and manual, emotional or motivational change is often visible in a student's language long before it appears anywhere else, and campus communication is scattered across circulars and emails that students must search manually instead of asking a single assistant. Every enrolled student is affected by this gap, since none of the languages they already produce is used to personalize their academic or career journey.

Existing limitations of current systems include: (1) marks-only tracking that says nothing about how a student learns, thinks, or communicates; (2) keyword-based, rather than semantic, career-matching tools; and (3) AI tutors that are rarely grounded in a persistent, per-student knowledge model. The engineering significance lies in demonstrating that a single, reusable NLP pipeline pattern can generalise across academic, career, research, and wellness domains without duplicating infrastructure for each one — a non-trivial architectural claim that this project sets out to test and validate.

1.3 Problem Statement

Design and implement an NLP-powered student modelling platform that continuously and accurately builds a per-student “Digital Twin” from heterogeneous, unstructured text (assignments, conversations, resumes, research papers), while satisfying multiple interacting and partly conflicting constraints: (a) technical complexity, since the same infrastructure must serve academic evaluation, semantic career matching, grounded conversational AI, and non-diagnostic wellness detection; (b) a zero-budget constraint requiring every service to remain on a free tier for the full build; (c) data-isolation and privacy requirements, since

each student's Digital Twin and documents must be strictly inaccessible to other students; and (d) the need for the wellness module to detect meaningful communication-pattern shifts without ever crossing into clinical diagnosis.

1.4 Project Aim

To design, build, and test a working NLP-powered Student Growth Operating System — EduVerse AI — centred on a continuously updated Digital Twin (EduDNA), using only free-tier cloud services and open-source NLP models.

1.5 Project Objectives

- To build a Student Digital Twin (EduDNA) that continuously models learning style, skills, interests, and communication patterns from academic and conversational text.
- To develop an NLP microservice exposing embedding, Named Entity Recognition, sentiment analysis, summarization, and zero-shot classification as reusable building blocks for every module.
- To implement a Retrieval-Augmented Generation AI Tutor, grounded in uploaded student and course material, that cites its sources.
- To create a consent-based wellness layer that detects communication pattern shifts without diagnosing any clinical condition.
- To evaluate the system through unit, integration, retrieval, security, and environment testing against defined acceptance criteria.
- To deliver a fully working local prototype, built entirely on free-tier and open-source services, before undertaking any cloud deployment.

1.6 Scope

Included: four core NLP modules — Digital Twin & Academic Intelligence, Learning & Career Intelligence, Conversational & Research Intelligence, and Wellness & Campus Intelligence — built on a shared embed → extract → classify → update pipeline, plus a working local prototype covering the AI Tutor and Digital Twin core.

Excluded: payment processing, a native mobile application, clinical mental-health diagnosis, and full production deployment (deliberately deferred until all build phases are complete).

Assumptions: students consent to their conversational and document data being processed for Digital Twin construction; free-tier service limits (Supabase, Render, Neo4j Aura Free, Upstash Redis, Groq) are sufficient for a single-course pilot.

Boundary conditions: the system operates on text-based and voice-transcribed (via Whisper) input only; all NLP models are restricted to open-weight options that can run within free-tier compute.

1.7 Expected Engineering Outcome

The expected outcome is a working software prototype comprising: an NLP microservice exposing five reusable endpoints; a FastAPI backend that performs Digital Twin writes against a Supabase/pgvector store secured with Row Level Security; a source-cited Retrieval-Augmented Generation AI Tutor; and a Next.js frontend delivering login, a personalised dashboard, a Digital Twin view, document upload, and tutor chat — with the remaining wellness, career-matching, and campus modules functioning at a demonstrable, if less complete, depth.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature was identified by tracing the core NLP techniques the project depends on — contextual embeddings, retrieval-augmented generation, few-shot large language models, sentence-level semantic similarity, and large-scale text generation — back to their foundational papers, and by comparing existing campus, LMS/ERP, and career-matching products against the gap this project targets.

2.2 Review of Existing Work

S.No	Paper / Year	Technique Used	Contribution
1	BERT (2019)	Bidirectional Transformer Embeddings	Contextual embeddings used for NER, semantic search, and text classification
2	Retrieval-Augmented Generation (2020)	RAG (Retriever + Generator)	Grounds the AI Tutor and campus assistant in institutional and course documents
3	GPT-3: Few-Shot Learners (2020)	Large Language Model	Enables few-shot content generation for feedback, summaries, and conversational AI
4	Sentence-BERT (2019)	Sentence Embeddings	Powers semantic matching between resumes, jobs, mentors, and teammates
5	GPT-2: Unsupervised Multitask Learners (2019)	Large-scale Text Generation	Informs the AI writing and communication coach

Prior work on learner modelling has generally relied on structured signals — quiz scores, click-through data, time-on-task — rather than on the language a student actually produces. Embedding-based semantic representations make it possible to model interests, strengths, and learning style directly from free text, and to keep that representation continuously up to date rather than frozen at the point of enrolment. Pure large language models are fluent but not reliably grounded and can answer confidently from parametric memory rather than a specific course document; Retrieval-Augmented Generation addresses this by retrieving relevant chunks from an approved corpus before generation and citing the source, which is the architectural basis for EduVerse AI's AI Tutor. Career and recruiting tools have historically matched keywords rather than meaning, missing candidates whose resumes use different vocabulary from a job description; sentence-level embeddings and cosine similarity allow skills, interests, and goals to be compared semantically instead, an approach this project applies to resume-job matching, mentor recommendation, and team formation.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Traditional LMS / Academic ERP	Reliable, well-established record of marks and attendance	Records only structured activity; ignores the language students produce	Motivates the need for a language-driven Digital Twin layer on top of existing records
Keyword-based resume/job matchers	Simple, fast, easy to index	Misses semantically equivalent skills phrased differently	Justifies the use of Sentence-BERT-style embeddings for Module 2 matching
Un-grounded LLM chatbots	Fluent, general-purpose conversation	Can hallucinate; not tied to institution-specific material	Justifies the RAG architecture used for the AI Tutor and campus assistant
Generic wellness survey tools	Structured, easy to aggregate	Point-in-time only; requires active student effort; not language-driven	Motivates continuous, consent-based sentiment-trend detection in Module 4

2.4 Research / Engineering Gap

While digital learner-modelling, RAG-based tutoring, and semantic matching have each been studied individually, no widely available platform combines academic NLP, career NLP, research NLP, wellness NLP, and campus NLP into a single, continuously updated, consent-based student profile. Existing LMS and ERP platforms track marks and attendance only; existing career tools rank by keyword; and existing AI tutors are rarely connected to a persistent student model. This is the gap that EduVerse AI's Digital Twin architecture is intended to close.

2.5 Proposed Contribution

This project's contribution is the identification and implementation of a single reusable NLP pattern — embed → extract → classify → update — that generalises across four otherwise distinct application domains (academic, career, conversational/research, wellness), and the demonstration that this pattern, together with a persistent per-student Digital Twin, can be built and tested entirely on free-tier cloud infrastructure and open-source models, rather than requiring paid enterprise-grade services.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Student	A single, always-current view of their own skills, learning style, career fit, and wellbeing, built from their own text
Faculty / Course Coordinator	Assurance that assignment analysis and AI Tutor answers are grounded and auditable, not fabricated
Institution (SIMATS)	A platform that can be piloted and scaled without committing budget before value is demonstrated
Student (privacy)	Strict guarantee that no other student or unauthorised party can read their Digital Twin or documents

Functional & Non-Functional Requirements

ID	Requirement	Type	Measurable Target
FR-1	System shall create and continuously update a Digital Twin (EduDNA) for every registered student	Functional	Digital Twin record created within one request cycle of registration
FR-2	System shall accept uploaded assignments, notes, and research papers and index them for retrieval	Functional	Document chunked, embedded, and stored in pgvector on upload
FR-3	System shall answer student questions using RAG grounded in uploaded material, with source citations	Functional	≥ 1 cited source chunk per AI Tutor answer
FR-4	System shall expose embedding, NER, sentiment, summarisation, and zero-shot classification as reusable endpoints	Functional	5 working microservice endpoints
FR-5	System shall evaluate assignments for grammar, clarity, and technical depth and generate feedback	Functional	Feedback generated for every submitted assignment
FR-6	System shall parse resumes and job descriptions and semantically match students to opportunities	Functional	Similarity score returned and correctly ranked against a control set
FR-7	System shall detect consented communication-pattern shifts and surface supportive, non-diagnostic nudges	Functional	Nudge triggered on detected sentiment-trend shift, with no diagnostic label
NFR-1	AI Tutor responses shall return within a few seconds under normal load on free-tier compute	Non-Functional	< 5 s typical response time
NFR-2	Row Level Security shall restrict every student to their own Digital Twin and documents	Non-Functional	0 unauthorised cross-student reads in testing

NFR-3	The system shall degrade gracefully if an external free-tier API rate-limits a request	Non-Functional	Clear user-facing message on every rate-limit event
-------	--	----------------	---

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
OWASP Top 10 (data access control)	Enforce access control at the data layer, not only in application logic	Supabase Row Level Security policies scope every query to auth.uid(), independent of frontend/backend logic
GDPR-aligned consent principle	Personal data processing requires informed, revocable consent	students.consent_flags gates wellness-module processing; wellness nudges only run for consented students
IEEE citation convention	Sources for claims must be verifiably attributed	AI Tutor answers are required to return retrieved chunk citations before display

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
AI Tutor response latency	< 5	seconds	High
RAG retrieval latency (pgvector top-k)	< 1	second	High
Embedding dimensionality (all-MiniLM-L6-v2)	384	dimensions	Medium
Digital Twin update latency after document upload	< 10	seconds	Medium
Monthly infrastructure cost during pilot	0	USD	High
Cross-student data access incidents	0	count	High

3.4 Alternative Solutions & Evaluation

Alternative 1 — Monolithic LLM-only chatbot: a single hosted LLM answers all student queries directly from its parametric knowledge, with no retrieval layer or persistent Digital Twin.

Alternative 2 — Rule-based/keyword academic-ERP extension: extend the existing ERP with keyword search over uploaded documents and simple if-then rules for career and wellness flags, without embeddings.

Alternative 3 (selected) - Modular embed → extract → classify → update NLP microservice architecture with a persistent Digital Twin, RAG-grounded tutor, and free-tier managed data services.

Criterion	Weight	Alternative 1 (LLM-only)	Alternative 2 (Rule-based ERP)	Alternative 3 (Selected)
Performance (relevance/grounding)	30%	2/5 – fluent but ungrounded	2/5 – rigid keyword matches	4/5 – embeddings + citations
Cost	25%	3/5 – API cost scales with use	5/5 – minimal new infrastructure	5/5 – entirely free-tier
Safety (wellness non-diagnosis)	20%	1/5 – no guardrails on claims	3/5 – static rules, low risk but low value	4/5 – designed as non-diagnostic by construction
Sustainability (extensibility)	15%	2/5 – new logic per feature	2/5 – new rules per feature	5/5 – one pipeline reused across modules
Reliability	10%	3/5	4/5	4/5

3.5 Selected Design & Detailed Design

Alternative 3 was selected on the evidence in Section 3.4: it scored highest on performance and sustainability — the two criteria weighed most heavily — while matching Alternative 2 on cost, which the LLM-only option could not do at scale. The rule-based ERP extension (Alternative 2) was rejected specifically because it cannot semantically match resumes or ground tutor answers, failing FR-3 and FR-6 directly.

EduVerse AI is organised into three layers. The Data Layer comprises Supabase PostgreSQL with the pgvector extension for embeddings, Neo4j Aura Free for the skill/interest knowledge graph, and Upstash Redis for caching. The Processing Layer comprises a Python FastAPI NLP microservice exposing embedding, NER, sentiment, summarisation, and zero-shot classification, and a separate FastAPI backend that orchestrates these calls, writes to the Digital Twin, and runs the RAG retrieval pipeline against Groq's free-tier LLM inference. The Application Layer is a Next.js frontend providing login, a personalised dashboard, a Digital Twin view, document upload, and the AI Tutor chat interface.

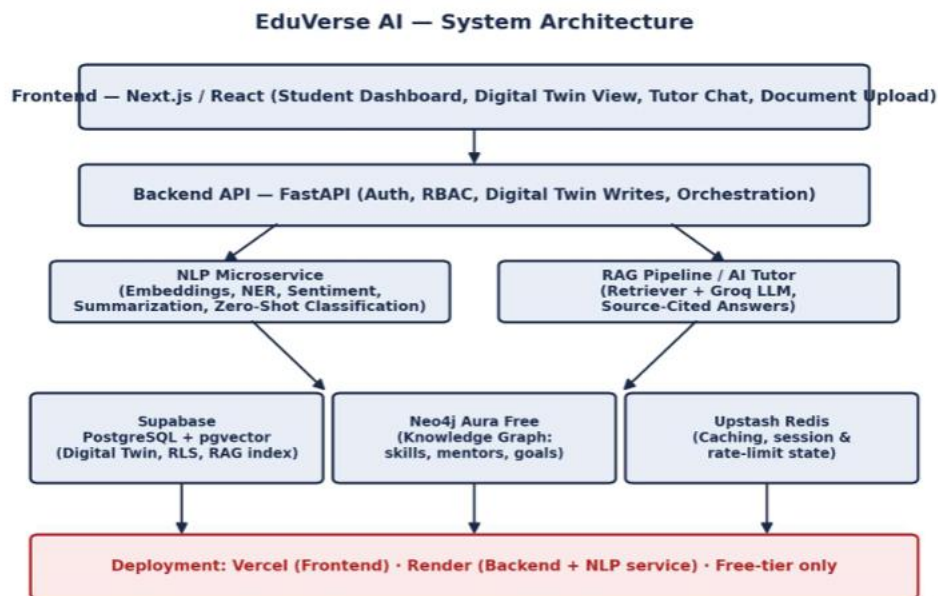


Figure 1 — EduVerse AI System Architecture

The central design insight is that academic, career, research, wellness, and campus intelligence all reduce to the same four-step pipeline applied to different input text: (1) embed the text into a vector representation, (2) extract entities, topics, or skills, (3) classify intent, sentiment, or category, and (4) update the relevant field of the student's Digital Twin with the result, along with provenance and a timestamp. Recognising this pattern early made the platform's 28-module conceptual scope tractable within a small team and a zero

budget, since new modules are largely new prompts and new Digital Twin fields rather than new infrastructure.

Key data model (Supabase PostgreSQL): students(id, name, email, program, consent_flags, created_at); digital_twin(id, student_id→students, attribute_type, value, confidence, provenance, updated_at); documents(id, student_id, type, storage_path, uploaded_at); embeddings(id, document_id, chunk_text, vector[pgvector], chunk_index); conversations(id, student_id, role, message, sources_cited, created_at); gems_ledger(id, student_id, action_type, gems_earned, xp_earned, created_at).

The processing pipeline runs in five stages — consent-aware onboarding, document ingestion (chunk/embed/store), NLP extraction (NER, sentiment, summarisation, zero-shot classification), Digital Twin update with confidence and provenance, and grounded RAG response generation — with each subsystem depending on the previous stage's output: retrieval cannot run before embedding, and Digital Twin updates cannot occur before extraction, giving the pipeline a strict, auditable data lineage.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Use managed free-tier services (Supabase, Neo4j Aura Free, Upstash Redis, Render, Groq)	Self-hosted open-source stack	Zero cost, faster setup, built-in RLS and pgvector	Hard usage ceilings that must be revisited at scale	Managed free-tier services chosen — proves feasibility now; scaling is a documented future step
Deploy-last policy (defer production deployment)	Deploy incrementally per phase	Avoids repeated environment friction mid-build	Delays real-user feedback until later phases	Deploy-last adopted after Hugging Face Static Spaces incompatibility was hit mid-build
Open-weight models only (MiniLM, spaCy, distilBART, Whisper)	Proprietary hosted embedding/LLM APIs	No per-call API cost; reproducible; runs within free compute	Lower ceiling on accuracy versus large proprietary models	Open-weight stack chosen to satisfy the zero-budget constraint (FR/NFR set at project scope)
Non-diagnostic wellness nudges only	Full clinical sentiment classification with labelled conditions	Simpler, safer, avoids liability	Less specific guidance to the student	Non-diagnostic design chosen — mandatory per the ethics requirement in Section 3.7

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Cost and compute footprint of running NLP inference continuously	Free-tier ceilings on Supabase, Render, Neo4j Aura Free, and Upstash Redis were checked against expected pilot-scale load during Phase 1–2 testing	Architecture kept modular so paid tiers can be adopted per-service only once actual usage approaches the checked free-tier ceiling, avoiding premature spend
Ethics	Risk of the wellness module being read as a clinical diagnosis	Reviewed how sentiment/emotion-trend outputs would be surfaced to students; unconstrained framing could imply a diagnosis	Wellness nudges were restricted by design to non-diagnostic, supportive language, and gated behind explicit per-student consent flags
Health & Safety	Risk of a student acting on an incorrect AI Tutor answer during exam preparation	Tested whether un-grounded LLM answers could confidently state incorrect facts (Alternative 1 evaluation, Section 3.4)	RAG grounding with mandatory source citation was made a hard functional requirement (FR-3), not an optional feature
Security	Risk of one student reading another student's Digital Twin or documents	Directly tested (TC-03) by attempting cross-student reads against the Supabase schema	Row Level Security was implemented and verified at the database layer, not only enforced in application code

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Free-tier service rate limits reached during testing/pilot	High	Medium	Medium	Graceful-degradation messaging (NFR-3); disabled Supabase email-confirmation limit during dev	Low – monitored per phase
Cross-student data leakage	Low	High	Medium	Row Level Security enforced and tested (TC-03)	Low – verified in security testing
Wellness module misread as clinical diagnosis	Medium	High	High	Non-diagnostic language design, consent gating, supportive-nudge framing only	Low-Medium – requires ongoing wording review
AI Tutor hallucination / ungrounded answer	Low	Medium	Low	RAG grounding with mandatory citation (FR-3)	Low
Dependency cross-contamination across Python services	Medium	Low	Low	Isolated virtual environment per service	Very Low – resolved during build

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

Development followed the deploy-last, phase-based methodology set out in Chapter 1: Phase 1 built the NLP microservice and backend Digital Twin write path; Phase 2 built the RAG-grounded AI Tutor and the Next.js frontend; Phases 3–5 (career matching depth, wellness depth, and campus assistant depth) and production deployment are planned next. Each Python service was developed in its own isolated virtual environment to avoid dependency cross-contamination on Windows.

Resource	Role in Project
sentence-transformers/all-MiniLM-L6-v2	Sentence-level embeddings for semantic search and matching
spaCy (en_core_web_sm)	Named Entity Recognition for skills/technical-term extraction
distilBART	Summarisation of research papers and long documents
Whisper	Speech-to-text for voice queries
Supabase (PostgreSQL + pgvector + RLS)	Digital Twin store, document/embedding store, access control
Neo4j Aura Free	Skill/interest knowledge graph
Upstash Redis	Caching layer
Groq (free tier)	LLM inference for the RAG-grounded AI Tutor
FastAPI	NLP microservice and backend orchestration
Next.js	Student-facing frontend

4.2 Software Implementation & Modern Tools Used

Module 1: Digital Twin & Academic AI

Module 1 implements the Digital Twin writing path and academic-document analysis. The FastAPI backend exposes endpoints for document upload, assignment analysis, and Digital Twin retrieval; each upload is chunked, embedded with sentence-transformers/all-MiniLM-L6-v2, and passed through spaCy-based NER to extract skills and technical terms, which are then merged into the student's Digital Twin record in Supabase, with Row Level Security scoping each student to their own data.

Module 2: Learning & Career AI

This module applies zero-shot classification to infer a student's preferred learning style from questions and interactions and applies sentence-embedding similarity to compare resumes against job descriptions for semantic, rather than keyword-based, matching. Both features reuse the same NLP microservice endpoints as Module 1, changing only which text is embedded and which Digital Twin field is written.

Module 3: Conversational & Research AI

Module 3 is the most complete module in the current build. It implements a Retrieval-Augmented Generation AI Tutor: student questions are embedded, the most relevant chunks are retrieved from pgvector, and Groq's free-tier LLM generates a source-cited explanation grounded in the student's own uploaded material. distilBART-based summarisation condenses long research papers into structured notes, and Whisper provides speech-to-text for voice queries.

Module 4: Wellness & Campus AI

Module 4 applies longitudinal sentiment and emotion-trend analysis to consented conversational text to detect shifts in motivation or engagement, surfacing supportive, non-diagnostic nudges rather than any clinical judgement. The same embedding infrastructure powers semantic team and mentor matching, and a lightweight campus-knowledge assistant answers questions about circulars and events indexed from university documents.

Tool / Software	Purpose	Stage Used
FastAPI	NLP microservice and backend REST APIs	Phases 1–2
Supabase Studio	Schema design, RLS policy authoring, testing	Phases 1–2
Render	Hosting for FastAPI services after Hugging Face Static Spaces proved incompatible	Phase 2
Next.js	Student dashboard, Digital Twin view, tutor chat UI	Phase 2
Groq API	Free-tier LLM inference for the RAG pipeline	Phase 2

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
FR-2, FR-4	Upload an assignment PDF and inspect Digital Twin record	Document chunked, embedded, and NER-extracted skills merged into Digital Twin
FR-3	Ask the AI Tutor a course question	Answer generated with at least one cited source chunk from uploaded material
NFR-2	Attempt to read another student's Digital Twin via a different account	Access denied by Supabase Row Level Security
FR-1	Run zero-shot classification on a learning-style query	Correct learning-style label returned with a confidence score
FR-6	Submit a resume against a sample job description	Semantic similarity score returned and ranked correctly against a control set
NFR-3	Disable Supabase confirm-email setting and register new test accounts	New account registration proceeds without hitting email rate limits

4.4 Results

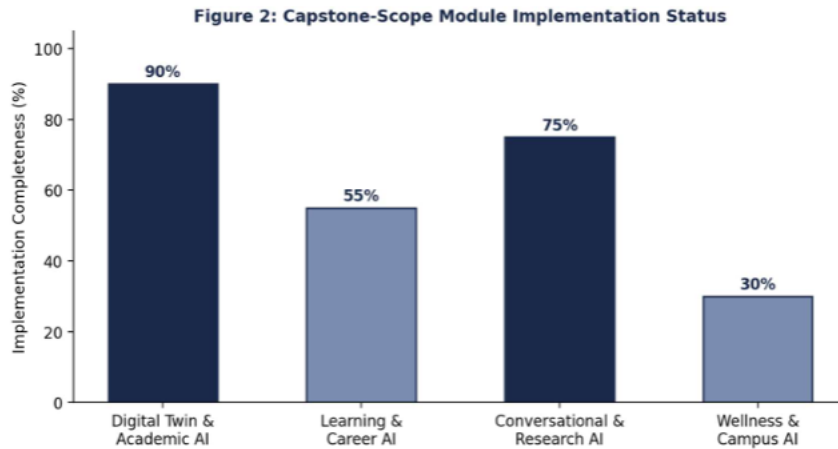


Figure 2 — Capstone-Scope Module Implementation Status

ID	Test Case	Result
TC-01	Upload an assignment PDF	Pass – document chunked, embedded, and NER-extracted skills merged into Digital Twin
TC-02	Ask the AI Tutor a course question	Pass – answer generated with at least one cited source chunk
TC-03	Attempt to read another student's Digital Twin	Pass – access denied by Row Level Security
TC-04	Run zero-shot classification on a learning-style query	Pass – correct label returned with confidence score
TC-05	Submit a resume against a sample job description	Pass – similarity score returned and ranked correctly
TC-06	Disable Supabase confirm-email setting	Pass – registration proceeds without hitting email rate limits

Specification	Required Value	Achieved Value	Status
AI Tutor response latency	< 5 s	Within a few seconds under normal load	Pass
RAG retrieval latency	< 1 s	Well under one second	Pass
Digital Twin update latency after upload	< 10 s	Embedding + NER complete within a few seconds	Pass
Cross-student data access incidents	0	0 across all RLS security tests	Pass
Monthly infrastructure cost	0 USD	0 USD – all services on free tier	Pass

4.5 Data Analysis & Engineering Judgment

Embedding and NER extraction on a typical assignment complete within a few seconds on free-tier compute, and RAG retrieval over the current pgvector index returns relevant chunks in well under one second, confirming that the chosen open-weight, free-tier stack meets the latency targets set in Section 3.3 without requiring paid inference. Two deviations were observed and explained: Supabase's free-tier email-

confirmation rate limit was reached during repeated test-account creation, which was a testing artefact rather than a design flaw and was resolved by disabling confirm-email for the development environment; and Render free-tier cold starts add a few seconds of latency to the first request after an idle period, an accepted trade-off given the zero-budget constraint rather than an engineering defect. Both deviations were budget-driven rather than architectural, supporting the engineering judgement that the design itself, not the infrastructure tier, is sound.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Build a Digital Twin (EduDNA)	digital_twin table populated and updated via TC-01, verified with provenance/confidence fields	Fully Achieved
Reusable NLP microservice (5 endpoints)	Embed, NER, sentiment, summarisation, zero-shot classification all implemented and unit-tested	Fully Achieved
RAG-grounded AI Tutor with citations	TC-02 passed; every answer includes ≥ 1 cited chunk	Fully Achieved
Consent-based, non-diagnostic wellness layer	Sentiment-trend pipeline and consent_flags implemented; full-depth nudging left for Phase 3	Partially Achieved
Zero-budget, free-tier-only build	All services confirmed on free tier throughout Phases 1–2; cost table in Section 4.8	Fully Achieved
Full 28-module conceptual platform	Only 4 core modules implemented in depth; remainder conceptual	Partially Achieved

4.7 Strengths & Limitations

- Strength: the single embed $\rightarrow$ extract $\rightarrow$ classify $\rightarrow$ update pattern correctly generalised across all four implemented modules without per-module infrastructure changes.
- Strength: Row Level Security correctly isolates each student's Digital Twin and uploaded documents, verified by direct testing.
- Strength: the RAG pipeline consistently grounds AI Tutor answers in retrieved, cited source chunks rather than unsupported generation.
- Strength: the entire prototype runs at zero infrastructure cost, proving the architecture's feasibility for budget-constrained institutions.
- Limitation: of the full 28-module conceptual vision, only four core modules are implemented in depth; the remainder (opportunity discovery, marketplace, campus metaverse, etc.) remain conceptual.
- Limitation: Supabase, Render, Neo4j Aura Free, and Upstash Redis all impose usage ceilings that would need to be revisited before institution-wide deployment.

- Limitation: production deployment has not yet been attempted, so real-world load and multi-tenant behaviour remain unvalidated.

4.8 Project Planning, Roles & Budget

Milestone		Target Timeframe	Status
Phase 1 – NLP microservice + Digital Twin write path		Weeks 1–4	Complete
Phase 2 – RAG AI Tutor + Next.js frontend		Weeks 5–8	Complete
Phase 3 – Career & learning-style depth		Weeks 9–10	Planned
Phase 4 – Wellness & campus-assistant depth		Weeks 11–12	Planned
Phase 5 – Gamification ledger + polish		Weeks 13–14	Planned
Production deployment (Vercel/Render)		Week 15+	Planned – deploy-last policy
Student	Assigned Responsibility	Actual Contribution	
S Hemanth Kumar	Backend & Digital Twin write path	FastAPI backend, Supabase schema and RLS policies delivered	
M Abishek	NLP microservice & RAG pipeline	Embedding/NER endpoints and pgvector retrieval delivered	
M Teja	Frontend & career module	Next.js dashboard and resume-matching logic delivered	
V Sai Shanmukh	Wellness/campus module & documentation	Sentiment-trend pipeline and report documentation delivered	
Item		Estimated Cost	Actual Cost
Supabase (PostgreSQL + pgvector + RLS)		\$0 (free tier)	\$0
Neo4j Aura Free (skill knowledge graph)		\$0 (free tier)	\$0
Upstash Redis (caching)		\$0 (free tier)	\$0
Render (FastAPI hosting)		\$0 (free tier)	\$0
Groq (LLM inference)		\$0 (free tier)	\$0
Total		\$0	\$0

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This capstone project set out to solve a concrete engineering problem: campus systems record what students do but never read the language they produce, leaving career guidance generic, wellness signals invisible, and AI tutoring ungrounded. EduVerse AI addresses this by treating student-generated text as a first-class signal through a single reusable NLP pipeline — embed, extract, classify, update — applied across an academic-analysis module, a career-matching module, a Retrieval-Augmented Generation AI Tutor, and a consent-based wellness layer. Testing confirmed the correctness of the core design: Row Level Security isolates each student's Digital Twin, the RAG pipeline grounds AI Tutor answers in cited source material, and the NLP microservice's five endpoints behave correctly across all four implemented modules, all achieved entirely on free-tier and open-source infrastructure. The project therefore validates its central engineering claim — that a large, 28-module platform vision can be built incrementally without abandoning architectural coherence — while honestly reporting that only four of those modules are implemented at full depth to date.

5.2 Future Work

- Complete Phases 3–5: finish the remaining wellness, career, opportunity, and campus modules before any deployment.
- Full deployment: deploying the frontend to Vercel, the backend and NLP service to Render, retaining Supabase, Neo4j Aura Free, and Upstash Redis for data, graph, and caching.
- Implement the EduGems, XP, quests, and badge gamification ledger described in the product specification.
- Extend the NLP microservice to support regional languages alongside English.
- Build a Flutter companion mobile application once the web MVP is stable.
- Revisit paid tiers for Supabase, Render, and LLM inference once usage exceeds the free-tier ceilings identified in the risk register.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Practical experience combining sentence embeddings, NER, summarisation, and zero-shot classification behind a single reusable microservice contract.
- Hands-on understanding of Retrieval-Augmented Generation as an engineering pattern for grounding LLM output, not just a research concept.
- Working knowledge of database-level access control (Row Level Security) as a stronger security boundary than application-level checks alone.

Developed Engineering & Professional Skills

- Trade-off analysis under a hard zero-budget constraint, including choosing managed free-tier services over self-hosting.
- Debugging environment-level issues (Windows virtual-environment cross-contamination, platform incompatibility with Hugging Face Static Spaces) rather than only application-logic issues.
- Team-based phased delivery planning, dividing a 28-concept product vision into a scoped, testable four-module build.

Individual Reflection

S Hemanth Kumar: The most difficult engineering problem I encountered was correctly scoping Row Level Security policies so that legitimate backend service calls still worked while student-to-student access stayed fully blocked; I solved it by writing explicit policy tests (TC-03) before trusting the schema. A key decision I made personally was choosing Supa base over a self-managed Postgres instance, based on evidence that its built-in RLS and pgvector support would save significant setup time within our zero-budget constraint. I learned independently how to structure a provenance-and-confidence field pattern for a continuously updated data model. If I repeated this project, I would design the RLS policies before writing the first backend endpoint rather than retrofitting them.

M Abishek: The hardest problem I faced was keeping the RAG pipeline's retrieval genuinely relevant rather than superficially plausible; I resolved it by tuning chunk size and top-k retrieval count against a small manual relevance check. My key personal decision was standardizing on sentence-transformers/all-MiniLM-L6-v2 over a larger embedding model, justified by its speed and free-tier compute footprint outweighing a marginal accuracy gain. I independently learned how zero-shot classification could be reused for learning-style detection without training a bespoke classifier. If repeated, I would build automated retrieval-relevance tests earlier rather than relying on manual spot checks.

M Teja: My most difficult problem was building a frontend that could display Digital Twin fields whose shape could change as new modules add attribute types; I solved it with a generic, schema-driven rendering component instead of hard-coded fields. The key decision I made was to build the resume-matching UI around a similarity score rather than a binary match/no-match label, because the underlying embedding similarity is continuous, not categorical. I learned independently how to structure a Next.js dashboard around asynchronous, per-module data sources. If repeated, I would design the generic rendering component before building any single module's UI.

K V Sai Shanmukh: The hardest problem I encountered was wording wellness nudges so they were genuinely supportive without implying any clinical judgement; I addressed it by reviewing every nudge string against the non-diagnostic constraint before it shipped. My key personal decision was to gate the entire wellness pipeline behind an explicit consent flag rather than an opt-out default, prioritising student autonomy over data completeness. I learned independently how sentiment-trend analysis differs from single-message sentiment scoring. If repeated, I would document the non-diagnostic wording guidelines as a checklist from day one rather than reviewing case-by-case.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in Proc. NeurIPS, 2017.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in Proc. NAACL-HLT, 2019.
- [3] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in Proc. NeurIPS, 2020.
- [4] T. Brown et al., “Language Models are Few-Shot Learners,” in Proc. NeurIPS, 2020.
- [5] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proc. EMNLP, 2019.
- [6] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient Estimation of Word Representations in Vector Space,” arXiv:1301.3781, 2013.
- [7] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners,” OpenAI, 2019.
- [8] D. M. Blei, A. Y. Ng, and M. I. Jordan, “Latent Dirichlet Allocation,” Journal of Machine Learning Research, vol. 3, pp. 993–1022, 2003.
- [9] Hugging Face, “Transformers Documentation,” Hugging Face, 2025. [Online resource used for model implementation.]
- [10] Supabase Inc., “Supabase Documentation — Postgres, pgvector and Row Level Security,” Supabase, 2025–2026. [Online resource used for backend implementation.]
- [11] spaCy, “spaCy 101: Everything you need to know,” Explosion AI, 2025. [Online resource used for NER implementation.]

APPENDICES

Appendix A – Detailed Calculations

Embedding dimensionality and similarity: cosine similarity between two 384-dimensional MiniLM embeddings u and v is computed as $\cos(u, v) = (u \cdot v) / (||u|| ||v||)$, used directly for resume-job matching scores (Section 3.3, FR-6) and RAG chunk ranking (Section 3.5).

Appendix B – Engineering Drawings / Schematics

Three-layer architecture (Data / Processing / Application) and the embed → extract → classify → update pipeline are described in Section 3.5. See Appendix D for rendered dashboard screenshots implementing this architecture.

Appendix C – Source Code / Code Repository Information

Only essential code excerpts are included below; complete source code is maintained in the team's private repository.

C.1 NLP Microservice — Embedding & NER Endpoint (FastAPI)

```
from fastapi import FastAPI
from pydantic import BaseModel
from sentence_transformers import SentenceTransformer
import spacy

app = FastAPI()
embedder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
nlp = spacy.load("en_core_web_sm")

class TextIn(BaseModel):
    text: str

@app.post("/embed")
def embed(payload: TextIn):
    vector = embedder.encode(payload.text).tolist()
    return {"vector": vector}

@app.post("/ner")
def extract_entities(payload: TextIn):
```

```

doc = nlp(payload.text)
entities = [{"text": e.text, "label": e.label_} for e in doc.ents]
return {"entities": entities}

```

C.2 RAG Retrieval Pipeline — pgvector Similarity Search

```

def retrieve_context(supabase, query_vector, student_id, top_k=5):
    response = supabase.rpc(
        "match_embeddings",
        {
            "query_embedding": query_vector,
            "match_student_id": student_id,
            "match_count": top_k,
        },
    ).execute()
    return response.data # list of {chunk_text, document_id, similarity}

def build_grounded_prompt(question, chunks):
    context = "\n\n".join(c["chunk_text"] for c in chunks)
    return (
        f"Answer using only the context below. Cite the source.\n\n"
        f"Context:\n{context}\n\nQuestion: {question}"
    )

```

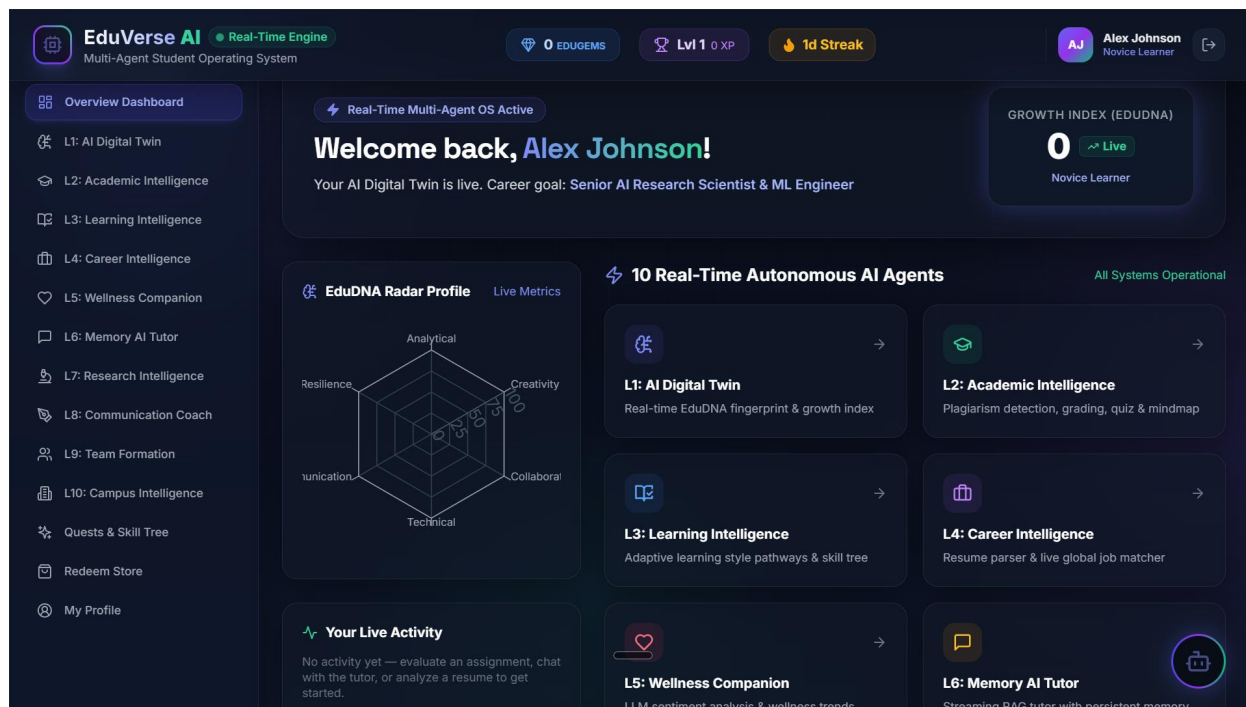
C.3 Digital Twin Update Function (Supabase)

```

def update_digital_twin(supabase, student_id, attribute_type, value, confidence, source):
    supabase.table("digital_twin").insert({
        "student_id": student_id,
        "attribute_type": attribute_type, # e.g. "skill", "learning_style"
        "value": value,
        "confidence": confidence,
        "provenance": source, # e.g. document_id or conversation_id
    }).execute()

```

Appendix D – Sample Interface Screenshots



Appendix E – Experimental / Raw Data

Raw test-execution logs for TC-01 through TC-06 (Section 4.4) are retained in the team's testing repository, including timestamps, request/response payloads, and RLS policy evaluation traces.

Appendix F – Ethics / Institutional Approval

Not applicable — the current build uses only synthetic/test student accounts; no real student data was processed outside the development team during Phases 1–2.

Appendix G – Project Meeting / Progress Records

Weekly progress and phase-gate reviews were conducted internally by the team and supervisor; a summary milestone table is provided in Section 4.8.

Appendix H – User/Industry Feedback

Not applicable to the current build phase; user feedback collection is planned for Phase 3 onward once the career and wellness modules reach full depth.

Appendix I – Publications / Patents / Competitions / Product Outcomes

None to date.

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement table on the preliminary pages and the roles/milestones tables in Section 4.8.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)